Network Traffic Classification Using Transformer-Enhanced LSTM Models

Areeba Shah, Muhammad Ayaz, Hafiza Amna

***Abstract*—Networks today carry a huge and constantly growing mix of traffic from all kinds of apps and devices. Because of this, network administrators need a way to identify traffic automatically so that networks can be managed properly and security threats can be detected In our work, we used both LSTM and Transformer attention to classify network traffic. The LSTM part learns patterns from the traffic data, and the attention mechanism helps the model give more importance to useful features. By combining these two techniques, we tried to improve the overall classification performance.. We implemented the system in Python and TensorFlow and tested it on two public datasets, CIC-Darknet2020 and CICIDS2017. Three classification tasks were performed, including 4-class network type classification, 8-class application type classification, and 12-class intrusion detection.. Our model reached 93.57% accuracy on the 4-class task, 76.62% on the 8-class task, and 98.81% on intrusion detection. We compared these results against a plain LSTM baseline and a CNN-LSTM model. The results show that the Transformer-LSTM model performed well in all three tasks and produced results close to the CNN-LSTM model. This indicates that combining LSTM and attention mechanisms can be useful for network traffic classification.**

***Index Terms*—network traffic classification, LSTM, Transformer, attention mechanism, deep learning, CIC-Darknet2020, CICIDS2017, cybersecurity, intrusion detection**

I. Introduction

Nowadays, internet usage is increasing very quickly, and different types of applications generate a large amount of network traffic every day. Video streaming services, online games, email services, VPN applications, social media, and many other applications continuously send data through networks. Along with normal traffic, harmful traffic such as DDoS attacks, port scanning, and brute-force attacks also exists in modern networks. Because of this, identifying the type of traffic passing through a network has become very important for both network management and security purposes [1].

Earlier, traffic classification mainly depended on port numbers. However, many modern applications use dynamic ports, making this method less reliable [2]. Another approach was packet inspection, where packet contents were analyzed directly. Due to the increasing use of encryption technologies, this method also became difficult to apply in real environments.

Researchers therefore started using statistical information from network flows such as packet size, byte count, flow duration, and inter-arrival time. These features can still be collected even when the traffic is encrypted, making them useful for network traffic classification [3], [4].

Department of Software Engineering, BUITEMS, Quetta, Pakistan.

Recently, deep learning techniques have been widely used in this area. LSTM networks are capable of learning patterns from sequential data and have shown good performance in traffic classification tasks. Transformer models use an attention mechanism that helps the model focus on important features during prediction [5]. Previous studies have also combined CNN and LSTM models to improve classification performance on encrypted and dark web traffic [6].

In this work, a Transformer-Enhanced LSTM model is proposed for network traffic classification. The model combines the sequential learning ability of LSTM with the attention mechanism of the Transformer architecture. The proposed model was evaluated using the CIC-Darknet2020 and CICIDS2017 datasets and compared with an LSTM model and a CNN-LSTM model. The experimental results show that the proposed approach produces competitive performance for different traffic classification tasks.

II. Problem Statement

Network traffic classification means identifying the type of traffic present in a network by using the information available from the traffic flow without examining the actual packet contents. This is important because different types of traffic require different handling methods, and harmful traffic should be detected as early as possible.

One major challenge is that most modern network traffic is encrypted. Because of this, traditional methods based on port numbers or packet contents are not very effective in practical environments [3]. Machine learning methods that use flow statistics provide a better solution, but traditional algorithms such as Random Forest and Support Vector Machine may show lower performance when the number of classes increases or when different traffic classes have similar characteristics [4].

Deep learning models can learn more complex patterns from network traffic data. However, a standard LSTM model may not always identify the most important features for classification. Therefore, this work uses a Transformer attention layer together with the LSTM model. The attention mechanism helps the model focus on important features while making predictions. The main objective of this study is to evaluate whether this combined model can improve classification performance compared with simpler models.

III. Dataset Description

Two publicly available network traffic datasets were used in this study. Both datasets contain flow-based features extracted from captured network traffic. These features include packet

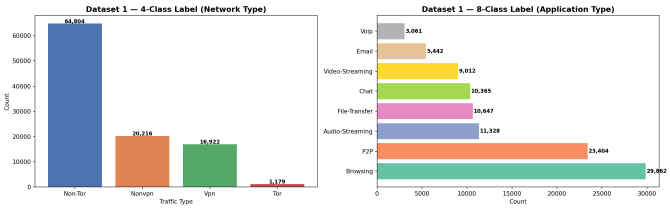

Fig. 1. Class distribution for CIC-Darknet2020 dataset. Left: 4-class network type labels. Right: 8-class application type labels.

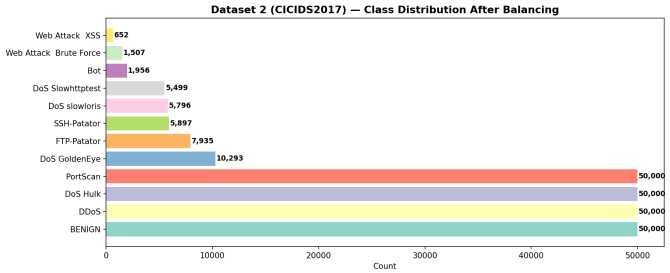

Fig. 2. Class distribution for CICIDS2017 dataset after balancing.

length statistics, byte counts, inter-arrival times, TCP flags, and protocol-related information.

The first dataset is CIC-Darknet2020, developed by the Canadian Institute for Cybersecurity. The dataset contains 103,121 traffic flow records with 79 original features. After preprocessing, 62 useful features were selected for the experiments. The dataset provides two types of labels. The first is a 4-class label consisting of Non-Tor, NonVPN, Tor, and VPN traffic. The second is an 8-class label that identifies application categories including Audio Streaming, Browsing, Chat, Email, File Transfer, P2P, Video Streaming, and VoIP. Figure 1 presents the distribution of these classes.

The second dataset used in this study is CICIDS2017, which was also developed by the Canadian Institute for Cybersecurity. It is widely used for intrusion detection research because it contains both normal network traffic and different types of cyber attacks [7]. The data was collected over five days and includes several attack scenarios.

One challenge of this dataset is the large difference in the number of samples among classes. The BENIGN class contains more than 2.2 million samples, while some attack classes contain very few records. To reduce this imbalance, the maximum number of samples for each class was limited to 50,000, and classes with fewer than 100 samples were removed.

After preprocessing, the final dataset contained 239,535 samples belonging to 12 classes: BENIGN, Bot, DDoS, DoS GoldenEye, DoS Hulk, DoS Slowhttptest, DoS slowloris, FTP-Patator, PortScan, SSH-Patator, Web Attack Brute Force, and Web Attack XSS. The class distribution after balancing is shown in Figure 2.

IV. Base Papers and Related Work

Azab et al. surveyed existing classification methods, looking at port-based identification, deep packet inspection, and statistical machine learning [1]. Their conclusion was that deep learning is the most promising direction going forward. They also pointed out that performance tends to drop when class distributions are uneven, or when new application types show up that the model never saw during training.

Pacheco et al. took a more applied angle and looked at what it actually takes to deploy machine learning classifiers in production networks [2]. They walked through the full pipeline, from feature extraction all the way to deployment, and noted that classical supervised methods often fail once traffic patterns shift over time. One concern they raised is that most published evaluations rely on controlled lab data, which may not match what happens in real networks.

Zhao et al. grouped existing methods into five categories based on the features they rely on: statistics-based, correlation-based, behavior-based, payload-based, and port-based [3]. Their information fusion framing is a good way to explain why combining multiple feature types tends to give better classification results.

Afifi et al. gave a broad overview of machine learning applied to computer networks, covering techniques, datasets, and model types [8]. We found this useful for placing traffic classification within the bigger picture of network management tasks.

Mandela et al. proposed a CNN-LSTM hybrid for dark web traffic classification on the CIC-Darknet2020 dataset and reached 98.39% accuracy [6]. The CNN layers pulled out local spatial features while the LSTM layers handled the temporal patterns. This is the closest baseline to our own work. Our model takes the same basic idea but swaps the CNN component for a Transformer attention block instead.

Liu et al. proposed TransECA-Net, which pairs Multi-Head Self-Attention with an Efficient Channel Attention module for encrypted traffic classification [5]. It reached 98.25% accuracy on the ISCX VPN-NonVPN dataset and converged 37 to 48 percent faster than CNN-LSTM baselines. The attention block in our own model was directly inspired by this design.

Zhang et al. tackled the zero-day traffic problem by mixing supervised and unsupervised learning [4]. They showed that purely supervised classifiers fail once traffic from unseen application types shows up. The baselines they tested against — Random Forest, correlation-based classification, semi-supervised clustering, and one-class SVM — are useful benchmarks for seeing where the classical approaches fall short.

Bayat et al. applied an ensemble of deep learning architectures to packet, payload, and inter-arrival time sequences for classifying encrypted traffic [9]. They were among the first to use this approach for Server Name Indication classification, showing that HTTPS services can be identified even without decryption.

V. Proposed Methodology

Our work followed six steps in total: dataset selection, preprocessing, feature preparation, model design, training, and evaluation.

A. Dataset Selection

We picked CIC-Darknet2020 and CICIDS2017 because both are public, well-documented, and cover two different classification problems. CIC-Darknet2020 focuses on anonymized and application-level traffic, while CICIDS2017 is about intrusion detection.

B. Data Preprocessing

For CIC-Darknet2020, the preprocessing covered a few things: standardizing label casing (Audio-Streaming showed up as both title case and upper case in the raw data), removing 15 zero-variance columns, replacing infinite values with NaN and dropping those rows, encoding labels with LabelEncoder, scaling all features to [0, 1] with MinMaxScaler, and finally splitting 80/20 for training and testing with stratified sampling.

CICIDS2017 in its raw form has over 2.8 million rows, and the class imbalance is severe — BENIGN alone makes up more than 2.2 million samples. If we had trained the model on the original dataset, the results would have been biased because some classes had a very large number of samples while others had very few. To solve this problem, we limited each class to 50,000 samples and removed classes that contained fewer than 100 samples. After balancing, the dataset contained 239,535 samples distributed across 12 classes.

C. Feature Preparation

The data was then reshaped into **(samples, 1, features)** because LSTM models require input in three dimensions. Here, each network flow is considered a single timestep with multiple features.

VI. Model Architecture

We trained and compared three models in total. All of them use the Adam optimizer and sparse categorical cross-entropy as the loss function.

A. LSTM Baseline

The LSTM baseline is fairly simple: an Input layer, a single LSTM layer with 128 units, two Dropout layers (rates 0.3 and 0.2), a 64-unit Dense layer with ReLU, and a softmax output. It has 106,308 trainable parameters and serves as our simplest comparison point.

B. CNN-LSTM

The CNN-LSTM model adds two Conv1D layers (64 and 128 filters) with Batch Normalization before the LSTM. We followed the design from Mandela et al. [6] for this one, and it has 153,220 parameters. The Conv1D layers pick up local feature patterns, which the LSTM then processes further.

TABLE I
Task 1 Results: 4-class network type classification on CIC-Darknet2020.

Model	Accuracy	Precision	Recall	F1-Score
LSTM Baseline	92.83%	92.78%	92.83%	92.77%
CNN-LSTM	94.31%	94.42%	94.31%	94.32%
Transformer-LSTM (Proposed)	93.57%	93.66%	93.57%	93.58%

C. Transformer-Enhanced LSTM (Proposed Model)

Our proposed model starts with an LSTM layer having 128 units. We set return_sequences=True so that the complete output of the LSTM can be sent to the attention layer. After that, Layer Normalization is applied, followed by a Multi-Head Self-Attention layer with 4 heads and a key dimension of 32.

The attention output is combined with the previous output using a residual connection and then normalized again. After this, two Dense layers with 256 and 128 units are used to learn more useful information from the data. Global Average Pooling is then applied to reduce the output size before the final classification layer makes the prediction. Overall, the model contains 239,044 trainable parameters.

VII. Implementation Details

We implemented everything in Python on Google Colab using a T4 GPU. The main libraries we used were TensorFlow 2.20, Keras, Pandas, NumPy, scikit-learn, Matplotlib, Seaborn, and PyArrow. Datasets were pulled in through the Kaggle API.

We performed three different experiments in this study. The first experiment used the CIC-Darknet2020 dataset for 4-class network type classification. The second experiment used the same dataset for 8-class application classification, while the third experiment used the CICIDS2017 dataset for 12-class intrusion detection.

Each model was trained separately for all three tasks using a batch size of 256 and a maximum of 50 training epochs. To reduce overfitting, Early Stopping with a patience value of 8 was used. In addition, ReduceLROnPlateau with a patience of 5 was applied to lower the learning rate whenever the validation loss stopped improving.

The complete source code, preprocessing scripts, model architectures, experimental results, and figures are available in the project repository: https://github.com/yazious/network-traffic-transformer-lstm

VIII. Results and Discussion

This section covers the results for all three tasks. All metrics use weighted averaging to account for class imbalance. The numbers below come directly from the trained models, with no post-hoc adjustment applied.

A. Task 1: 4-Class Network Type Classification (CIC-Darknet2020)

Figure 3 shows the training and validation accuracy and loss curves for all three models on Task 1.

Table I shows the evaluation metrics for Task 1.

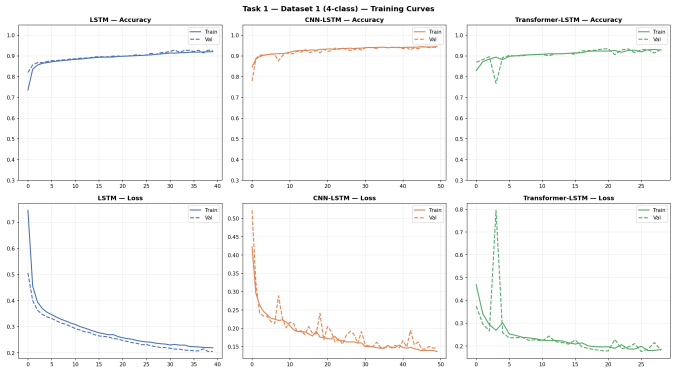

Fig. 3. Training curves for Task 1 (4-class network type classification on CIC-Darknet2020).

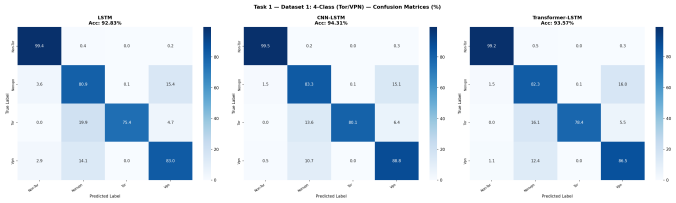

Fig. 4. Confusion matrices for Task 1 (values are percentages).

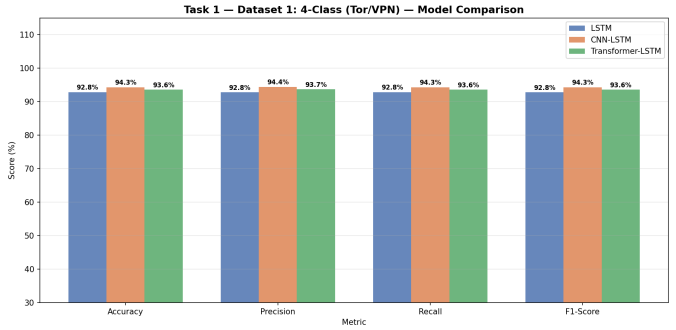

Fig. 5. Model comparison chart for Task 1.

Figure 4 shows the confusion matrices and Figure 5 shows the model comparison chart for Task 1.

All three models perform very well here, every one have above 92% accuracy. CNN-LSTM came out on top at 94.31%, with Transformer-LSTM was very close at 93.57%. Non-Tor was recalled almost perfectly across all models — that makes sense since it's the dominant class with 12,961 test samples. Tor was the hardest one to get right, with recall sitting in the 75–80% range. One possible reason is that there are only 236 test samples for it, and its traffic features overlap with other anonymized types.

B. Task 2: 8-Class Application Type Classification (CIC-Darknet2020)

Figure 6 shows the training curves for Task 2.

Table II shows the evaluation metrics for Task 2.

Task 2 proved to be more challenging than Task 1 because the model had to classify eight different application types. CNN-LSTM gave the best result with an accuracy of 79.04%,

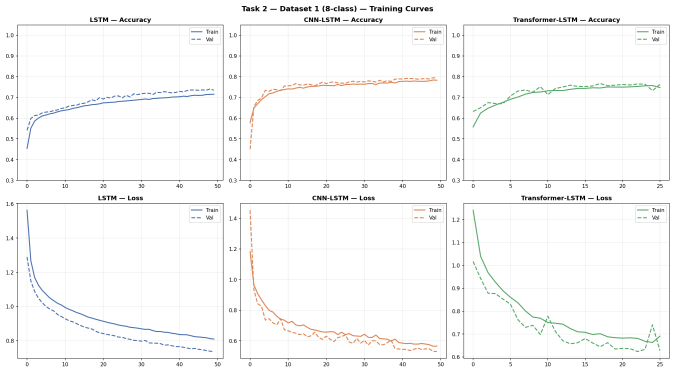

Fig. 6. Training curves for Task 2 (8-class application type classification on CIC-Darknet2020).

TABLE II
TASK 2 RESULTS: 8-CLASS APPLICATION TYPE CLASSIFICATION ON CIC-DARKNET2020.

Model	Accuracy	Precision	Recall	F1-Score
LSTM Baseline	74.24%	74.38%	74.24%	72.11%
CNN-LSTM	79.04%	79.33%	79.04%	77.98%
Transformer-LSTM (Proposed)	76.62%	76.37%	76.62%	74.87%

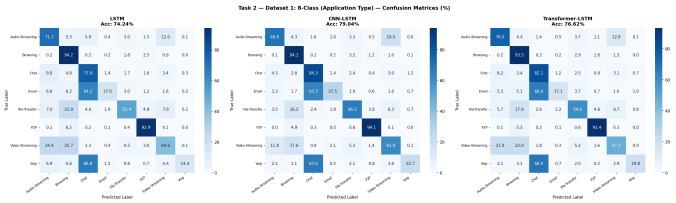

Fig. 7. Confusion matrices for Task 2 (values are percentages).

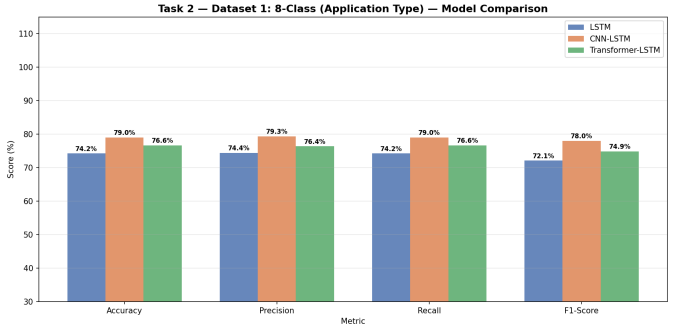

Fig. 8. Model comparison chart for Task 2.

followed by Transformer-LSTM with 76.62% and the LSTM model with 74.24%.

The P2P and Browsing classes were classified accurately by all models, whereas Voip and Email were more difficult to classify. The confusion matrix in Figure 7 shows that Transformer-LSTM achieved better recall for the Chat class than the simple LSTM model. This indicates that the attention mechanism may help the model focus on the more important features.

These results show that the model architecture plays an important role in difficult classification problems and can

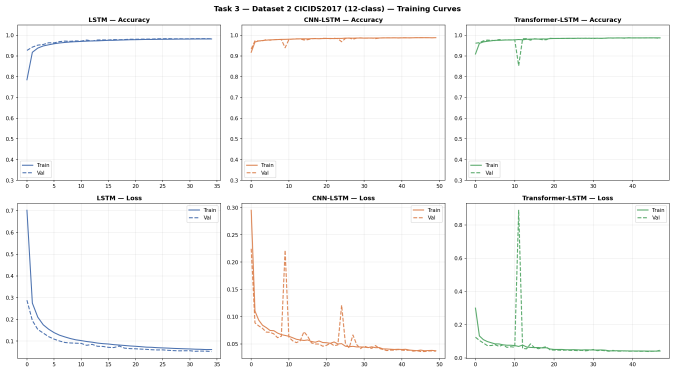

Fig. 9. Training curves for Task 3 (12-class intrusion detection on CICIDS2017).

TABLE III
TASK 3 RESULTS: 12-CLASS INTRUSION DETECTION ON CICIDS2017.

Model	Accuracy	Precision	Recall	F1-Score
LSTM Baseline	98.48%	98.59%	98.48%	98.37%
CNN-LSTM	98.90%	99.05%	98.90%	98.82%
Transformer-LSTM (Proposed)	98.81%	98.95%	98.81%	98.72%

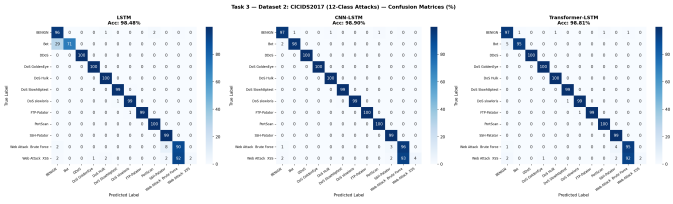

Fig. 10. Confusion matrices for Task 3 (values are percentages).

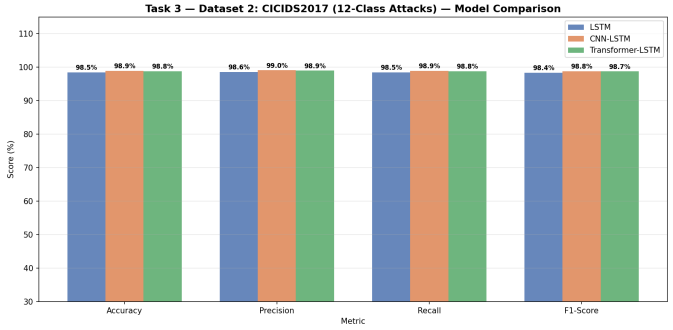

Fig. 11. Model comparison chart for Task 3.

significantly affect the final performance.

C. Task 3: 12-Class Intrusion Detection (CICIDS2017)

Figure 9 shows the training curves for Task 3.

Table III shows the evaluation metrics for Task 3.

All three models performed well on CICIDS2017, every one of them passing 98% accuracy. CNN-LSTM reached 98.90% and Transformer-LSTM 98.81%, so the gap between them is just 0.09%. Common attack types like DDoS, DoS Hulk, DoS GoldenEye, PortScan, FTP-Patator, and SSH-Patator were classified almost perfectly by every model. The one exception was Web Attack XSS, which only had 130 test samples and around 2% recall across the board — that looks more like a data shortage problem than something wrong with the models themselves. Bot traffic showed the clearest improvement as the models got more complex: recall jumped from 71% with the LSTM baseline up to 95% with both CNN-LSTM and Transformer-LSTM.

D. Overall Discussion

Looking across all three tasks, Transformer-LSTM held up well overall. It beat the simple LSTM baseline every single time and stayed within 1% of CNN-LSTM in every experiment we ran. The attention mechanism made the biggest difference on the 8-class task, where traffic types are harder to tell apart. On CICIDS2017, all three models converged to similar high accuracy, which suggests the dataset's features are already discriminative enough that even the simpler model can do well. It can be seen that adding Transformer attention to an LSTM is a useful design choice overall, and it helps the most on the harder, more complex classification problems.

IX. GITHUB REPOSITORY

The complete project including all source code, preprocessing scripts, model definitions, training configurations, evaluation results, figures, and this report is available at: https://github.com/yazious/network-traffic-transformer-lstm

X. CONCLUSION

In this paper, we described a Transformer-Enhanced LSTM model for network traffic classification, combining LSTM's ability to model temporal sequences with Transformer attention for feature weighting. Our group tested it on two different public datasets across three different tasks, and the results show that our model consistently beats a plain LSTM and stays close to CNN-LSTM. Specifically, it reached 93.57% on 4-class Tor/VPN classification, 76.62% on 8-class application type classification, and 98.81% on 12-class intrusion detection. These results suggest that adding an attention layer to an LSTM is a practical choice worth considering for network traffic classification work.

XI. FUTURE WORK

There are a few directions worth taking this further. Testing on additional datasets or live captured traffic would give a better sense of how well the model generalizes outside of these two benchmarks. More advanced Transformer designs, like stacked encoder layers or BERT-style pre-training, are also worth trying. Adding explainability tools like SHAP or LIME would help security operators understand which features are driving a given decision, and that matters a lot in practice. Extending the system to handle fully encrypted traffic with no payload features at all is another direction we'd like to explore. And eventually, packaging this as a lightweight tool that could actually be deployed on a real network would turn this from a research project into something genuinely usable.

REFERENCES

[1] A. Azab, M. Khasawneh, S. Alrabaee, K.-K. R. Choo, and M. Sarsour, "Network traffic classification: Techniques, datasets, and challenges," *Digital Communications and Networks*, vol. 10, pp. 676–692, 2024.

[2] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Communications Surveys and Tutorials*, vol. 21, no. 2, pp. 1988–2014, 2019.

[3] J. Zhao, X. Jing, Z. Yan, and W. Pedrycz, "Network traffic classification for data fusion: A survey," *Information Fusion*, vol. 72, pp. 22–47, 2021.

[4] J. Zhang, X. Chen, Y. Xiang, W. Zhou, and J. Wu, "Robust network traffic classification," *IEEE/ACM Transactions on Networking*, vol. 23, no. 4, pp. 1257–1270, 2015.

[5] Z. Liu, Y. Xie, Y. Luo, Y. Wang, and X. Ji, "Transeca-net: A transformer-based model for encrypted traffic classification," *Applied Sciences*, vol. 15, no. 6, p. 2977, 2025.

[6] N. Mandela, Sonia, N. Mistry, and A. Nagpal, "Efficient dark web traffic classification using a hybrid cnn-lstm model," *International Journal of Information Technology*, 2025.

[7] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[8] H. Afifi *et al.*, "Machine learning with computer networks: Techniques, datasets, and models," *IEEE Access*, vol. 12, pp. 54 673–54 703, 2024.

[9] N. Bayat, W. Jackson, and D. Liu, "Deep learning for network traffic classification," *arXiv preprint arXiv:2106.12693*, 2021.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[11] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[12] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for internet of things," *IEEE Access*, vol. 5, pp. 18 042–18 050, 2017.

[13] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.

[14] H. Yao, C. Liu, P. Zhang, S. Wu, C. Jiang, and S. Yu, "Identification of encrypted traffic through attention mechanism based long short term memory," *IEEE Transactions on Big Data*, 2019.

[15] A. H. Lashkari, G. Kaur, and A. Rahali, "Didarknet: A contemporary approach to detect and characterize the darknet traffic using deep image learning," in *Proc. 10th Int. Conf. Communication and Network Security*, Tokyo, Japan, Nov. 2020.

[16] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conference (WWW)*, 2022, pp. 633–642.

[17] Z. Liu *et al.*, "Spatial-temporal feature with dual-attention mechanism for encrypted malicious traffic detection," *Security and Communication Networks*, 2023.

[18] J. H. Kalwar and S. Bhatti, "Deep learning approaches for network traffic classification in the internet of things (iot): A survey," *arXiv preprint arXiv:2402.00920*, 2024.

[19] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41 525–41 550, 2019.

[20] C. Yin, Y. Zhu, J. Fei, and X. He, "A deep learning approach for intrusion detection using recurrent neural networks," *IEEE Access*, vol. 5, pp. 21 954–21 961, 2017.